

Enhanced Corrosion Prediction with OCP and LSV Features (NaOH)

This notebook extends the basic NaOH Tafel-based model by incorporating features extracted from Open Circuit Potential (OCP) and Linear Sweep Voltammetry (LSV) data to improve prediction accuracy.

```
!pip install optuna
```

```
Collecting optuna
  Downloading optuna-4.6.0-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.12/dist-
Collecting colorlog (from optuna)
  Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.12/di
Downloading optuna-4.6.0-py3-none-any.whl (404 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 404.7/404.7 kB 6.8 MB/s eta 0:00:00
Downloading colorlog-6.10.1-py3-none-any.whl (11 kB)
Installing collected packages: colorlog, optuna
Successfully installed colorlog-6.10.1 optuna-4.6.0
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import optuna
import warnings
warnings.filterwarnings('ignore')
optuna.logging.set_verbosity(optuna.logging.WARNING)
```

```
plt.style.use('default')
sns.set_palette("husl")
```

✓ Load Data

```
# Load cleaned NaOH data
df_tafel = pd.read_csv('/content/sample_data/cleaned_NAOH_tafel.csv')
print(f"Tafel data shape: {df_tafel.shape}")
print(df_tafel.head())

# Load OCP data
df_ocp = pd.read_csv('/content/sample_data/cleaned_NAOH_OCP.csv')
print(f"\nOCP data shape: {df_ocp.shape}")
print(df_ocp.head())

# Load LSV data
df_lsv = pd.read_csv('/content/sample_data/cleaned_NAOH_LSV.csv')
print(f"\nLSV data shape: {df_lsv.shape}")
print(df_lsv.head())
```

Tafel data shape: (17, 9)

	Sample	NaOH_M	Inhibitor_ppm	Temp_C	E_corr_V	j_corr_A_cm2	\
0	1.75M, 50PPM, 30	1.75	50	30	-0.818033	1.765620e-07	
1	1M, 125PPM, 60	1.00	125	60	-0.779654	7.656550e-06	
2	1M, 125PPM, 30	1.00	125	30	-0.758955	4.989650e-07	
3	1.75M, 50PPM, 60	1.75	50	60	-1.167750	3.869720e-03	
4	1M, 200PPM, 45	1.00	200	45	-1.161890	7.854070e-03	

	I_corr_A	CR_mm_yr	PR_ohm
0	1.765620e-07	0.002052	147583
1	7.656550e-06	0.088969	3403.32
2	4.989650e-07	0.005798	52223.5
3	3.869720e-03	44.965900	6.73374
4	7.854070e-03	91.263800	3.31773

OCP data shape: (17, 7)

	Sample	Time_s	OCP_V	Potential_V	NaOH_M	\
0	1.75M, 50PPM, 30	0.140234	-0.432190	0.352905	1.75	
1	1M, 125PPM, 60	0.141231	-0.274445	-0.177185	1.00	
2	1M, 125PPM, 30	0.134245	-0.490540	-0.285431	1.00	
3	1.75M, 50PPM, 60 DEGREES	0.137739	-0.351501	-0.259033	1.75	
4	1M, 200PPM, 45 DEGREES	0.129754	-0.342926	0.507935	1.00	

	Inhibitor_ppm	Temp_C
0	50	30
1	125	60
2	125	30
3	50	60
4	200	45

LSV data shape: (20893, 7)

	Sample	Potential_V	Current1_A	Current2_A	NaOH_M	\
0	1.75M,50PPM, 30	-1.92978	-0.000157	0.000157	1.75	
1	1.75M,50PPM, 30	-1.92734	-0.000133	0.000133	1.75	
2	1.75M,50PPM, 30	-1.92490	-0.000120	0.000120	1.75	
3	1.75M,50PPM, 30	-1.92245	-0.000109	0.000109	1.75	
4	1.75M,50PPM, 30	-1.92001	-0.000102	0.000102	1.75	

	Inhibitor_ppm	Temp_C
0	50	30
1	50	30
2	50	30
3	50	30
4	50	30

✓ Feature Extraction from OCP and LSV

```
# Extract OCP features (summary data)
def extract_ocp_features(df_ocp):
    features = []
    for sample in df_ocp['Sample'].unique():
        sample_data = df_ocp[df_ocp['Sample'] == sample]

        # Use the summary values
        ocp_value = sample_data['OCP_V'].iloc[0] if 'OCP_V'
        time_value = sample_data['Time_s'].iloc[0]

        features.append({
            'Sample': sample,
            'OCP_value': ocp_value,
            'OCP_time': time_value
        })
    return pd.DataFrame(features)

# Extract LSV features (curve analysis)
def extract_lsv_features(df_lsv):
    features = []
    for sample in df_lsv['Sample'].unique():
        sample_data = df_lsv[df_lsv['Sample'] == sample].copy()

        # Sort by potential
        sample_data = sample_data.sort_values('Potential_V')

        # Basic features from LSV curve
        potential_range = sample_data['Potential_V'].max() - sample_data['Potential_V'].min()
        current_max = sample_data[['Current1_A', 'Current2_A']].max().max()
        current_min = sample_data[['Current1_A', 'Current2_A']].min().min()

        # Simple slope estimation (linear approximation)
        mid_idx = len(sample_data) // 2
        anodic_data = sample_data.iloc[mid_idx:]
        cathodic_data = sample_data.iloc[:mid_idx]
```

```

if len(anodic_data) > 1 and len(cathodic_data) > 1:
    try:
        anodic_slope = np.polyfit(anodic_data['Poten
        cathodic_slope = np.polyfit(cathodic_data['I
    except:
        anodic_slope = cathodic_slope = 0
else:
    anodic_slope = cathodic_slope = 0

features.append({
    'Sample': sample,
    'LSV_potential_range': potential_range,
    'LSV_current_max': current_max,
    'LSV_current_min': current_min,
    'LSV_anodic_slope': anodic_slope,
    'LSV_cathodic_slope': cathodic_slope
})
return pd.DataFrame(features)

# Extract features
ocp_features = extract_ocp_features(df_ocp)
lsv_features = extract_lsv_features(df_lsv)

print("OCP Features:")
print(ocp_features.head())
print("\nLSV Features:")
print(lsv_features.head())

```

OCP Features:

	Sample	OCP_value	OCP_time
0	1.75M,50PPM, 30	-0.432190	0.140234
1	1M, 125PPM, 60	-0.274445	0.141231
2	1M,125PPM,30	-0.490540	0.134245
3	1.75M,50PPM, 60 DEGREES	-0.351501	0.137739
4	1M , 200PPM, 45 DEGREES	-0.342926	0.129754

LSV Features:

	Sample	LSV_potential_range	LSV_current_max \
0	1.75M,50PPM, 30	2.99805	0.001118
1	1M, 125PPM, 60	2.99804	0.000489
2	1M,125PPM,30	2.99805	0.000895
3	1.75M, 50PPM, 60 DEGREES	2.99805	0.100000
4	1M ,200PPM ,45 DEGREES	2.99805	0.100000

	LSV_current_min	LSV_anodic_slope	LSV_cathodic_slope
0	-0.000157	0.000343	0.000033
1	-0.000127	0.000270	0.000042
2	-0.000015	0.000339	0.000007
3	-0.099997	0.055529	0.160719
4	-0.099997	0.064792	0.155714

Merge Features with Tafel Data

```
# Merge features
df_enhanced = df_tafel.copy()
df_enhanced = df_enhanced.merge(ocp_features, on='Sample', how='left')
df_enhanced = df_enhanced.merge(lsv_features, on='Sample', how='left')

print(f"Enhanced dataset shape: {df_enhanced.shape}")
print(df_enhanced.head())

# Check for missing values
print("\nMissing values:")
print(df_enhanced.isnull().sum())

# Fill missing values
df_enhanced = df_enhanced.fillna(df_enhanced.median(numeric_
```

Enhanced dataset shape: (17, 16)

	Sample	NaOH_M	Inhibitor_ppm	Temp_C	E_corr_V	j_corr_A_cm2	\
0	1.75M, 50PPM, 30	1.75	50	30	-0.818033	1.765620e-07	
1	1M, 125PPM, 60	1.00	125	60	-0.779654	7.656550e-06	
2	1M, 125PPM, 30	1.00	125	30	-0.758955	4.989650e-07	
3	1.75M, 50PPM, 60	1.75	50	60	-1.167750	3.869720e-03	
4	1M, 200PPM, 45	1.00	200	45	-1.161890	7.854070e-03	

	I_corr_A	CR_mm_yr	PR_ohm	OCP_value	OCP_time	LSV_potential_range	\
0	1.765620e-07	0.002052	147583	-0.432190	0.140234	2.99805	
1	7.656550e-06	0.088969	3403.32	-0.274445	0.141231	2.99804	
2	4.989650e-07	0.005798	52223.5	NaN	NaN	NaN	
3	3.869720e-03	44.965900	6.73374	NaN	NaN	NaN	
4	7.854070e-03	91.263800	3.31773	NaN	NaN	NaN	

	LSV_current_max	LSV_current_min	LSV_anodic_slope	LSV_cathodic_slope
0	0.001118	-0.000157	0.000343	0.000033
1	0.000489	-0.000127	0.000270	0.000042
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

Missing values:

Sample	0
NaOH_M	0
Inhibitor_ppm	0
Temp_C	0
E_corr_V	0
j_corr_A_cm2	0
I_corr_A	0
CR_mm_yr	0
PR_ohm	0
OCP_value	15
OCP_time	15
LSV_potential_range	15
LSV_current_max	15

```

LSV_current_min      15
LSV_anodic_slope      15
LSV_cathodic_slope    15
dtype: int64

```

✓ Feature Engineering and Preparation

```

# Prepare features
feature_cols = ['NaOH_M', 'Inhibitor_ppm', 'Temp_C', 'OCP_v',
                'LSV_potential_range', 'LSV_current_max', 'I',
                'LSV_anodic_slope', 'LSV_cathodic_slope']
X_base = df_enhanced[feature_cols].copy()

# Polynomial features
X_base['NaOH_sq'] = X_base['NaOH_M'] ** 2
X_base['Inhibitor_sq'] = X_base['Inhibitor_ppm'] ** 2
X_base['Temp_sq'] = X_base['Temp_C'] ** 2

# Logarithmic features
X_base['Log_NaOH'] = np.log1p(X_base['NaOH_M'])
X_base['Log_Inhibitor'] = np.log1p(X_base['Inhibitor_ppm'])
X_base['Log_Temp'] = np.log(X_base['Temp_C'])

# Inverse features
X_base['Inv_NaOH'] = 1 / X_base['NaOH_M']
X_base['Inv_Inhibitor'] = 1 / (X_base['Inhibitor_ppm'] + 1)
X_base['Inv_Temp'] = 1 / X_base['Temp_C']

# Interaction terms
X_base['NaOH_Inhibitor'] = X_base['NaOH_M'] * X_base['Inhibitor_ppm']
X_base['NaOH_Temp'] = X_base['NaOH_M'] * X_base['Temp_C']
X_base['Inhibitor_Temp'] = X_base['Inhibitor_ppm'] * X_base['Temp_C']

# Standardize
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_base), columns=X_base.columns)

# Target
y = df_enhanced['CR_mm_yr']

print(f"Features: {list(X_scaled.columns)}")
print(f"Dataset shape: {X_scaled.shape}")

# Save enhanced scaler
import joblib
import os
os.makedirs('./cleaned data naoh', exist_ok=True)
joblib.dump(scaler, './cleaned data naoh/enhanced_feature_scaler.pkl')

```

```

Features: ['NaOH_M', 'Inhibitor_ppm', 'Temp_C', 'OCP_value', 'OCP_time', 'LSV_pc']
Dataset shape: (17, 22)
['./cleaned data naoh/enhanced_feature_scaler.pkl']

```

✓ Model Training and Comparison

```

## Model Training and Comparison
# Optuna objectives

from xgboost import XGBRegressor

def objective_ridge(trial):
    alpha = trial.suggest_float('alpha', 1e-4, 10.0, log=True)
    model = Ridge(alpha=alpha)
    return -cross_val_score(model, X_train, y_train, cv=3, scoring='neg_mean_squared_error')

def objective_rf(trial):
    n_estimators = trial.suggest_int('n_estimators', 50, 200)
    max_depth = trial.suggest_int('max_depth', 3, 15)
    model = RandomForestRegressor(n_estimators=n_estimators, max_depth=max_depth)
    return -cross_val_score(model, X_train, y_train, cv=3, scoring='neg_mean_squared_error')

def objective_xgb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'random_state': 42,
        'verbosity': 0
    }
    model = XGBRegressor(**params)
    return -cross_val_score(model, X_train, y_train, cv=3, scoring='neg_mean_squared_error')

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
cv = KFold(n_splits=3, shuffle=True, random_state=42))

# Optimize models
study_ridge = optuna.create_study(direction='minimize')
study_ridge.optimize(objective_ridge, n_trials=20)
ridge = Ridge(alpha=study_ridge.best_params['alpha'])

study_rf = optuna.create_study(direction='minimize')
study_rf.optimize(objective_rf, n_trials=15)
rf = RandomForestRegressor(**study_rf.best_params, random_state=42)

study_xgb = optuna.create_study(direction='minimize')
study_xgb.optimize(objective_xgb, n_trials=20)

```

```

xgb_params = study_xgb.best_params.copy()
xgb_params['random_state'] = 42
xgb_params['verbosity'] = 0
xgb_model = XGBRegressor(**xgb_params)

# Models
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': ridge,
    'Random Forest': rf,
    'XGBoost': xgb_model,
    'MLP': MLPRegressor(hidden_layer_sizes=(32,), alpha=0.001)
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    cv_r2 = cross_val_score(model, X_train, y_train, cv=cv,
                             test_r2 = r2_score(y_test, model.predict(X_test)))

    results.append({
        'Model': name,
        'CV R²': cv_r2,
        'Test R²': test_r2
    })

results_df = pd.DataFrame(results)
print("Model Performance with Enhanced Features (NaOH):")
print(results_df.to_string(index=False))

# Select best model
best_idx = results_df['Test R²'].idxmax()
best_model_name = results_df.loc[best_idx, 'Model']
best_model_score = results_df.loc[best_idx, 'Test R²']

print(f"\nBest Model: {best_model_name} (Test R² = {best_model_score})")

# Save best model
best_model = models[best_model_name]
joblib.dump(best_model, './cleaned data naoh/enhanced_best_model.pkl')
print("Enhanced best model saved.")

```

Model Performance with Enhanced Features (NaOH):

	Model	CV R²	Test R²
Linear Regression	-233.463317	-191.110732	
Ridge	-30.186173	-16.167091	
Random Forest	-19.159572	-6.224058	
XGBoost	-16.405975	-4.051123	
MLP	-0.408020	-0.351805	

Best Model: MLP (Test R² = -0.352)

Enhanced best model saved.

